Week05 – Math and Logic Instruction, MQTT + Ubidots, BLE Remote
PC ____ รหัส _____ ชื่อ-สกุล _____

1. Math Instruction (TS Emulator)
   1.1 เลือกโจทย์จาก."คำถามในการทดลอง ตอนที่ 1"

2. Logic Instruction (TS Emulator)
   2.1 เขียนโปรแกรมตามโปรแกรม "Lab42.Asm" คอมไพล์และโหลดโปรแกรมด้วย TS-Emulator ทดสอบการทำงานของโปรแกรมแบบ Single Step สังเกตการเปลี่ยนแปลงข้อมูลในหน่วยความจำที่เกี่ยวข้องและบันทึกผลการรันโปรแกรมลงในตารางบันทึกผล
   2.2 เลือกโจทย์จาก."คำถามในการทดลอง ตอนที่ 2"

3. HEX, BCD (TS Emulator)
   3.1 เลือกโจทย์จาก."คำถามในการทดลอง ตอนที่ 3"

Final

4. MQTT
   4.1 อ่านค่า DHT-22 แล้วส่งไปยัง MQTT Broker ทุกๆ 2 วินาที
   4.2 เพิ่มเติมให้ทำงานรับคำสั่งเพื่อควบคุม LED จำนวน 4 ตัว
   4.3 เพิ่มเติมให้รับค่าสวิตซ์ กำหนด
   - หากกด SW1 แจ้ง Overheat Alarm
   - หากกด SW2 แจ้ง Intruders Alarm

5. Ubidots
   5.1 อ่านค่า DHT-22 แล้วส่งไปยัง MQTT Broker ทุกๆ 2 วินาที ให้ GUI แสดงค่าเป็นกราฟ
   5.2 เพิ่มเติมให้ทำงานรับคำสั่งเพื่อควบคุม LED จำนวน 4 ตัว ให้ GUI แสดงค่าเป็นปุ่มควบคุม
   5.3 เพิ่มเติมให้รับค่าสวิตซ์ ให้ GUI แสดงค่าเป็นแลมป์อินดิเคเตอร์ กำหนด
   - หากกด SW1 แจ้ง Overheat Alarm
   - หากกด SW2 แจ้ง Intruders Alarm

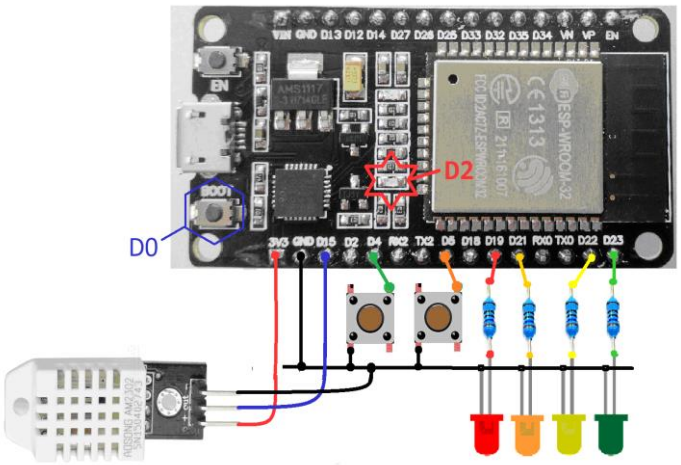
6. BLE Remote
   6.1 ติดตั้ง Dabble บน Smart Phone: https://thestempedia.com/product/dabble/
   6.2 ใช้ ESP32 BLE ในการสื่อสารกับ Mobile Application เพื่อควบคุมการปิดเปิด LED จำนวน 8 ตัว

Math Instruction	Logic Instruction	HEX2BCD BCD2HEX	MQTT	Ubidots	BLE Remote

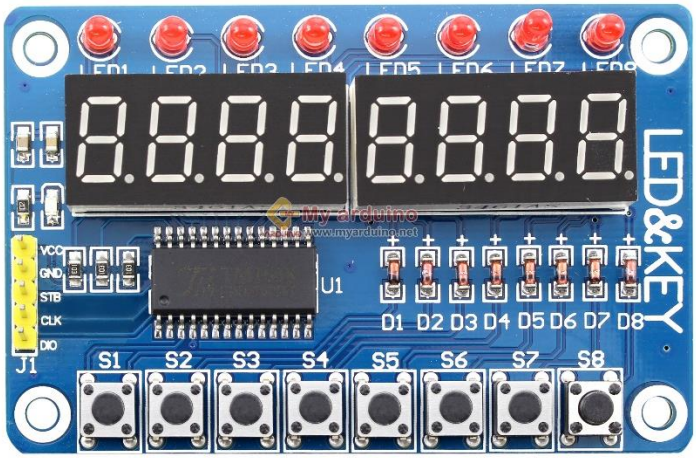
ตารางบันทึกผล

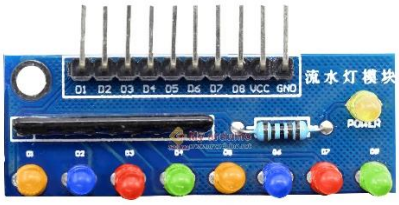
Assembly instruction		PC	DPTR	Acc	PSW	C	AC	21H	24H	8888H
Item	Initial	0000								
1	ORG 0000H									
2	MOV A, #0FEH									
3	ANL A, #0F0H									
4	ORL A, #005H									
5	RR A									
6	INC A									
7	RL A									
8	SETB Acc.4									
9	DEC A									
10	SWAP A									
11	CRL C									
12	ADD A, #0FFH									
13	RLC A									
14	SETB 21H									
15	ANL A, 24H									
16	MOV DPTR, #8888H									
17	MOVX @DPTR, A									
18	SJMP $									
19	END									

MQTT



Mobile Application with BLE to ESP32





Note

1/7. MCS-51 Assembly Code:

```
; ==== Lab41.Asm ====

HResult  EQU     20H
LResult  EQU     21H

         ORG     0000H
         MOV     HResult,#00        ; Clear Result
         MOV     LResult,#00

         CLR     C
         MOV     A,R0
         ADDC    A,LResult
         MOV     LResult,A          + R0
         MOV     A,#00
         ADDC    A, HResult
         MOV     HResult,A

         CLR     C
         MOV     A,R1
         ADDC    A,LResult
         MOV     LResult,A          + R1
         MOV     A,#00
         ADDC    A,HResult
         MOV     HResult,A

         CLR     C
         MOV     A,R2
         ADDC    A,LResult
         MOV     LResult,A          + R2
         MOV     A,#00
         ADDC    A,HResult
         MOV     HResult,A

         JMP     $
         END
```

```
; ==== Lab42.Asm ====

         ORG     0000H              ; 1
         MOV     A,#0FEH            ; 2
         ANL     A,#0F0H            ; 3
         ORL     A,#005H            ; 4

         RR      A                  ; 5
         INC     A                  ; 6
         RL      A                  ; 7
         SETB    Acc.4              ; 8
         DEC     A                  ; 9
         SWAP    A                  ; 10
         CLR     C                  ; 11
         ADD     A,#0FFH            ; 12
         RLC     A                  ; 13

         SETB    21H                ; 14
         ANL     A,24H              ; 15
         MOV     DPTR,#8888H        ; 16
         MOVX    @DPTR,A            ; 17

         SJMP    $                  ; 18
         END                        ; 19
```

25 + 46 + 87

```
 Δ Δ Δ +     Δ Δ Δ +     Δ Δ Δ +
   2 5         4 6         8 7
 Δ Δ Δ       Δ Δ Δ       Δ Δ Δ
```

```
MOV   R0,#0
MOV   R1,#0
MOV   R2,#0
```

2/7. เนื้อหา ทฤษฎีที่เกี่ยวข้อง

1. การใช้คำสั่งการบวก การลบ การคูณ และการหาร

ADD A, Source
ADDC A, Source
DA A
SUBB A, Source
MUL AB ; Result BA
DIV AB ; Result A, Remain B

50
05
01 90
B A

B = 0B H A 02 H

DIV AB
A = 05
B = 01

2. การใช้คำสั่งการคำนวณเชิงตรรกะ

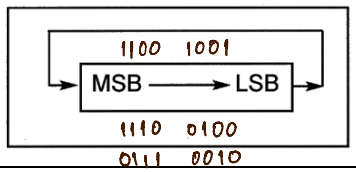
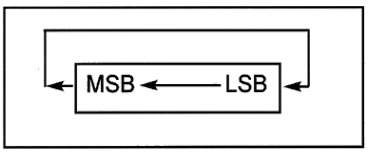
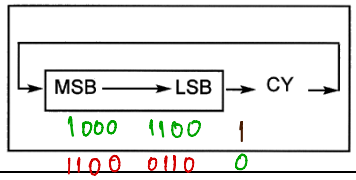
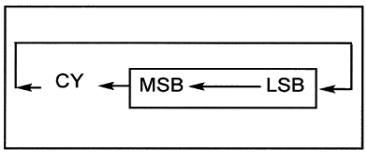
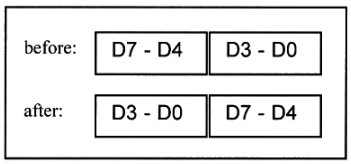
ส้มไม่ ← **AND A, Source**
เปิดไม่ ← **ORL A, Source**
เหมือนได้ 0 ← **XRL A, Source**
CPL A

XOR-1 = CPL

0⊕1 = 1
1⊕1 = 0
0⊕0 = 0
1⊕0 = 1

3. การใช้คำสั่งเกี่ยวกับการหมุนบิต

RR A ;rotate right A
RL A ;rotate left A
RRC A ;rotate right with carry
RLC A ;rotate left with carry
SWAP A ; Swap
XCH A,B ; Exchange

RR A		RL A	
RRC A		RLC A	
SWAP A		XCH A, B แลกค่า A = 12 B = 28	**Before A=AAH, B=BBH** **After A=BBH, B=AAH** XCH A = 28 B = 12

RR 4 ครั้ง = SWAP

A = 56
Swap A
A = 65

4. การใช้คำสั่งเพื่อศึกษาการแปลงระหว่างเลขฐานสิบหกกับลขฐานสิบ

HEX2BCD ➔ FDH (253D)
Step1: FDH/0AH ➔ Result =19H Remain =03H
Step2: 19H/0AH ➔ Result =02H Remain =05H
Step3: 02H/0AH ➔ Result =00H Remain =02H

A FD
10 256
10 25 6
2 → 5

BCD2HEX ➔ 45H (2DH)
High Nipple * 10 + Low Nipple = Result
04H * 0AH + 05H = 2DH

unpack = 02 H
= 05 H
= 06 H

BCD — Packed BCD ⇒ 12 H 48 H 52
\ Unpack BCD ⇒ 01 H 04 H 05 H
02 H 02 H 02 H

A5 H — 04 ×10
\ 05
×10

3/7. คำถามในการทดลอง

<u>ตอนที่ 1</u> คำสั่งการบวก ลบ คูณ และหาร

1.1 โปรแกรม “Lab41.Asm” เป็นโปรแกรมสำหรับรวมค่า R0, R1 และ R2 ทดสอบการทำงานด้วย TS-Emulator

- ก่อนรันโปรแกรมกำหนดค่า **R0=R1=R2=0FFH**
- ผลลัพท์เก็บที่แอดเดรส 20H(MSB)=02H ,21H(LSB)=FDH {FF+FF+FF=02FD}

1.2 จากโปรแกรม “Lab41.Asm” ให้นักศึกษาปรับปรุงโปรแกรมเพื่อรวมค่าต่างๆแล้วเก็บผลลัพธ์ที่ได้ในหน่วยความจำภายนอกที่ 9500H-9501H

1.2.1 รวมค่าใน R0-R7 แบบไบนารี่

1.2.2 รวมค่าในหน่วยความจำภายในตำแหน่งที่ 40H-47H แบบไบนารี่

1.2.3 รวมค่าในหน่วยความจำภายนอกตำแหน่งที่ 9000H-9007H แบบไบนารี่

1.3 ให้นักศึกษาเขียนโปรแกรมเพื่อดำเนินการลบเลขฐานสิบหก จำนวน 2 ตัว ดังต่อไปนี้

ตัวตั้งกำหนดไว้ก่อนรันโปรแกรมที่หน่วยความจำภายนอกตำแหน่ง 8800H

ตัวลบกำหนดไว้ก่อนรันโปรแกรมที่หน่วยความจำภายในตำแหน่ง 31H

คัดลอกข้อมูลจากตำแหน่ง 8800H มาเก็บไว้ที่หน่วยความจำภายในตำแหน่ง 21H

ดำเนินการลบ แล้วเก็บผลลัพธ์ที่ได้ไว้ในหน่วยความจำตำแหน่ง 40H และ 41H

1.4 ให้นักศึกษาเขียนโปรแกรมบวกเลขฐานสอง จำนวน 4 ไบต์ แบบไบนารี่ โดยมีข้อกำหนด ดังนี้

ตัวตั้ง = หน่วยความจำภายนอกที่ตำแหน่ง 9001H (MSB) - 9004H (LSB)

ตัวบวก = หน่วยความจำภายนอกที่ตำแหน่ง 9011H (MSB) - 9014H (LSB)

ผลลัพธ์ = หน่วยความจำภายนอกที่ตำแหน่ง 9020H (MSB) - 9024H (LSB)

หมายเหตุ: การบวกต้องคำนึงถึงตัวทดระหว่างไบต์ และต้องมีการบันทึกผลลัพธ์ไบต์ที่ 5 ด้วย

1.5 เขียนโปรแกรมเพื่อทำการคูณเลข 2 จำนวน ตามข้อกำหนด ดังนี้

ตัวตั้งเป็นเลข 2 ไบต์ที่เก็บใน R4:R3 (R4 =MSB, R3=LSB)

ตัวคูณเป็นเลข 1 ไบต์ที่เก็บใน R6

ผลลัพธ์ที่ได้ไปเก็บในหน่วยความจำภายนอก โดยให้ค่าไบต์ต่ำเริ่มที่ตำแหน่ง 9000H และให้โปรแกรมเริ่มต้นที่แอดเดรส 8200H

<u>ตอนที่ 2</u> คำสั่งการคำนวณเชิงตรรกะและคำสั่งเกี่ยวกับการหมุนบิต

2.1 การคำนวณเชิงตรรกะส่วนใหญ่ใช้เพื่อต้องการตรวจสอบระดับบิตหรือการจัดการระดับบิตในกรณีที่รีจีสเตอร์ไม่สามารถจัดการระดับบิตได้

- กรณีที่ต้องการให้รีจีสเตอร์ TMOD เฉพาะ 4 บิตล่างเป็น 0101B จะใช้ชุดคำสั่งดังนี้

```
ANL     TMOD,#11110000B
ORL     TMOD,#00000101B
```

- กรณีที่ต้องการตรวจสอบว่าให้รีจีสเตอร์ Acc มีค่าเท่ากับ 10110101B หรือไม่

```
XRL     A,#10110101B
```

ผลการทำงานชุดคำสั่งนี้

- ถ้าผลลัพธ์ Acc = 00000000B แสดงว่าก่อนการทำคำสั่งนี้ค่า A=10110101B
- ถ้าผลลัพธ์ Acc ≠ 00000000B แสดงว่าก่อนการทำคำสั่งค่า A≠10110101B

2.2 กำหนดให้ Register B มีค่าอยู่ก่อนแล้ว ให้นักศึกษาทำการนับจำนวนเลข 1 ในค่าใน Register B เก็บผลลัพธ์ที่ได้ใน R7

<u>**เช่น**</u> B=F7H ➔ 1111 0111 Run แล้วได้ R7=01H

2.3 กำหนดให้ Register DPTR มีค่าอยู่ก่อนแล้ว ให้นักศึกษาทำการนับจำนวนเลข 0 ในค่าใน Register DPTR เก็บผลการนับแบบ BCD ที่รีจีสเตอร์ B

<u>**เช่น**</u> DPTR=1234H ➔ 0001 0010 0011 0100 Run แล้วได้ B=11H (11 ตัว)

2.4 ให้นักศึกษาเขียนโปรแกรมเพื่อทำการนับว่าค่าในหน่วยความจำตำแหน่ง 40H – 7FH มีจำนวนบิตที่เป็น 1 กี่บิต กำหนดหน่วยความจำสำหรับเก็บผลลัพธ์ด้วยตัวเอง

<u>**{เมื่อโปรแกรมทำงานจะให้ผลลัพธ์สูงสุดเท่าใด, ควรเก็บผลลัพธ์กี่ไบต์}**</u>

<u>ตอนที่ 3</u> การแปลงเลขฐานสิบหกเป็นฐานสิบ

3.1 เขียนนักศึกษาโปรแกรมเพื่อรวมเลขแบบ BCD ระหว่าง DPTR R0 และ R1 รวมเสร็จแล้วให้เก็บค่าผลลัพธ์ที่หน่วยความจำภายในตำแหน่งใดๆก็ได้ {ระหว่าง 30H-7FH }

<u>เช่น</u> DPTR=1234H, R0=00H และ R1=11H

Result = 1234+00+11 = 1245H เป็นต้น

3.2 ใส่ค่าเริ่มต้นลงในหน่วยความจำในข้อมูลตำแหน่ง 8500H เพื่อเป็นค่าที่ต้องการแปลงจากไบนารี่(HEX) เป็นเลขบีซีดี(BCD)

3.3 ใส่ค่าเริ่มต้นลงในรีจีสเตอร์ R1 เพื่อเป็นค่าที่ต้องการแปลงจากเลขบีซีดี(BCD)เป็นเลขไบนารี่(HEX) โดยค่านี้ต้องเป็นเลข BCD เท่านั้น

3.4 ให้นักศึกษาเขียนโปรแกรมแปลงเลขบีซีดีขนาด 8 บิท ที่เก็บไว้ในรีจีสเตอร์ R1 เป็นเลขไบนารี่ เก็บไว้ในหน่วยความจำภายในแอดเดรส 30H

00 00 00 40 – 7F

Nibble = Ø กี่ตัว

3/7. คำถามท้ายการทดลอง

Q1. ให้นักศึกษาเขียนโปรแกรมเพื่อทำการนับว่าค่าในหน่วยความจำตำแหน่ง 40H – 7FH มีจำนวนไบต์ที่เป็น FF กี่ไบต์ {เมื่อโปรแกรมทำงานจะให้ผลลัพธ์สูงสุดเท่าใด, ควรเก็บผลลัพธ์กี่ไบต์}

```
        ORG     0000H
        MOV     B,#00           ; Result
        MOV     R0,#40H         ; Start
        MOV     R1,#40H         ; 7F-40+1

LOOP:   MOV     A,@R0
        XRL     A,#0FFH
        JNZ     xLoop
        INC     B
xLoop:  INC     R0
        DJNZ    R1,LOOP
        JMP     $
        END
```

CJNZ 'not Equl Jump'

XOR เหมือนได้ Ø

40H - 7FH มีจำนวน 64 ไบต์

คำตอบมากสุด คือ 64D, 40H → 1ไบต์

Q2. ให้นักศึกษาเขียนโปรแกรมเพื่อทำการนับว่าค่าในหน่วยความจำตำแหน่ง 40H – 7FH มีจำนวนนิบเบิล (4Bit)ที่เป็น F กี่นิบเบิล {เมื่อโปรแกรมทำงานจะให้ผลลัพธ์สูงสุดเท่าใด, ควรเก็บผลลัพธ์กี่ไบต์}

```
        ORG     0000H
        MOV     B,#00           ; Result
        MOV     R0,#40H         ; Start
        MOV     R1,#40H         ; 7F-40+1

HNib:   MOV     A,@R0           กข
        SWAP    A               บก
        ANL     A,#0FH          0F
        XRL     A,#0FH          0ก
        JNZ     LNib
        INC     B
LNib:   MOV     A,@R0
        CPL     A
        ANL     A,#0FH
        JNZ     XLoop
        INC     B
XLoop:  INC     R0
        DJNZ    R1,HNib
        JMP     $
        END
```

40H - 7FH มีจำนวน 64 ไบต์

มีจำนวน 128 นิบเบิล

คำตอบมากสุด คือ 128D, 80H → 1ไบต์

Q3. ให้นักศึกษาเขียนโปรแกรมเพื่อทำการนับว่าค่าในหน่วยความจำตำแหน่ง 40H – 7FH มีจำนวนบิตที่เป็น 1 กี่บิต {เมื่อโปรแกรมทำงานจะให้ผลลัพธ์สูงสุดเท่าใด, ควรเก็บผลลัพธ์กี่ไบต์}

ไบต์ →	ORG 0000H
	MOV DPTR,#0000 ; Result
	MOV R0,#40H ; Start
หา loop →	MOV R1,#40H ; 7F-40+1
LOOP:	MOV R2,#8 ; 8 Bit → loop ใน
	MOV A,@R0
_BitC:	RLC A
	JNC _Zero
	INC DPTR
_Zero:	DJNZ R2,_BitC
	INC R0
	DJNZ R1,LOOP
	JMP $
	END

C 1100 1101
1 1001 10 10

40H - 7FH มีจำนวน 64 ไบต์
มีจำนวน 512 บิต
คำตอบมากสุด คือ 512D, 0200H → 2ไบต์

```
Loop:  MOV   A,@R0
       MOV   C,ACC.0
       CALL  ADDXXX
       MOV   C,ACC.1
       CALL  ADDXXX
       :
       MOV   C,ACC.7
       CALL  ADDXXX
       DJNZ  R1,LOOP
       JMP   $

ADDXXX: JMP_EXIT
        INC DPTR

← EXIT: RET
```

Q4. ให้นักศึกษาเขียนโปรแกรมเพื่อทำการนับว่าค่าในหน่วยความจำตำแหน่ง 40H – 7FH มีจำนวนบิตที่เป็น 0 กี่บิต ให้ตอบเป็นเลข BCD {เมื่อโปรแกรมทำงานจะให้ผลลัพธ์สูงสุดเท่าใด, ควรเก็บผลลัพธ์กี่ไบต์}

```
        ORG     0000H
        MOV     DPTR,#0000      ; Result
        MOV     R0,#40H         ; Start
        MOV     R1,#40H         ; 7F-40+1

LOOP:   MOV     R2,#8   ; 8 Bit
        MOV     A,@R0
_BitC:  RLC     A
        JC      _Zero
        CALL    BCDInc_DPTR
_Zero:  DJNZ    R2,_BitC
        INC     R0
        DJNZ    R1,LOOP
        JMP     $

BCDInc_DPTR:
        CLR     C
        MOV     A,DPL
        ADDC    A,#01
        DA      A
        MOV     DPL,A
        MOV     A,DPH
        ADDC    A,#00
        DA      A
        MOV     DPH,A
        RET

        END
```

40H - 7FH มีจำนวน 64 ไบต์

มีจำนวน 512 บิต

คำตอบเป็นเลข BCD คือ 0512H → 2ไบต์

Q5. ให้นักศึกษาเขียนโปรแกรมแปลงเลขไบนารี่ ขนาด 8 บิท ที่เก็บไว้ในหน่วยความจำ 8500H

- เป็นเลข Unpacked BCD ขนาด 3หลัก เก็บไว้ในหน่วยความจำ R5, R6, R7
- เป็นเลข Packed BCD ขนาด 2ไบต์/3หลัก เก็บไว้ในหน่วยความจำ DPTR

```
        ORG     0000H
        MOV     DPTR,#8500H
        MOVX    A,@DPTR         ; 123D
        CALL    Hex_Acc2BCD_DPTR
        JMP     $

Hex_Acc2BCD_DPTR:
        MOV     B,#10   ;
        DIV     AB      ; A=12, B=03
        MOV     R7,B    ; 03
        MOV     B,#10   ;
        DIV     AB      ; A=01, B=02
        MOV     R6,B    ; 02
        MOV     R5,A    ; 01

        MOV     DPH,R5 ; 01
        MOV     A,R6
        SWAP    A
        ORL     A,R7
        MOV     DPL,A   ; 23
        RET

        END
```

EAH → 234D

7BH → 123D

HEX2BCD → FDH (253D)

Step1: FDH/0AH → Result =19H Remain =03H
Step2: 19H/0AH → Result =02H Remain =05H
Step3: 02H/0AH → Result =00H Remain =02H

BCD2HEX → 45H (2DH)

High Nipple * 10 + Low Nipple = Result
04H * 0AH + 05H = 2DH

Q6. กำหนดให้ป้อนค่าใน Register B ก่อนรันโปรแกรม และให้นักศึกษาเขียนโปรแกรมเพื่อบวกค่าจาก 0 ถึงค่าที่เก็บใน Register B กำหนดหน่วยความจำสำหรับเก็บผลลัพธ์ด้วยตัวเอง

<u>{เมื่อโปรแกรมทำงานจะให้ผลลัพธ์สูงสุดเท่าใด, ควรเก็บผลลัพธ์กี่ไบต์}</u>

<u>เช่น</u> Register B = 0FH

0 + 1 + 2 + … + 9 + A + B + C + D + E + F = 78H เป็นต้น

```
        ORG     0000H
        MOV     DPTR,#0000H ; Result

LOOP:   MOV     A,B
        JZ      END_ADD_LOOP
        CLR     C
        ADDC    A,DPL
        MOV     DPL,A
        MOV     A,DPH
        ADDC    A,#0
        MOV     DPH,A
        DEC     B
        JMP     LOOP
END_ADD_LOOP:

        SJMP    $
        END
```

ค่า B ที่ทำให้คำตอบสูงสุด คือ FF(255)

ผลการบวก 0+…+255 = 32640(7F80H)

→2ไบต์

Q7. กำหนดให้ป้อนค่าใน Register B ก่อนรันโปรแกรม และให้นักศึกษาเขียนโปรแกรมเพื่อรวมตั้งแต่ค่าที่เก็บใน Register B จนถึง 99H

- หากค่าที่ป้อน มากกว่า 99H ให้คำตอบเป็น F ทุกนิบเบิล
- กำหนดหน่วยความจำสำหรับเก็บผลลัพธ์ด้วยตัวเอง
  <u>{เมื่อโปรแกรมทำงานจะให้ผลลัพธ์สูงสุดเท่าใด, ควรเก็บผลลัพธ์กี่ไบต์}</u>

<u>เช่น</u>	Register B = 9AH	→ <Over 99H>	= FFFFF…
	Register B = 99H	→ 99H	= 99H
	Register B = 98H	→ 98H + 99H	= 131H

```
        ORG     0000H
        MOV     DPTR,#0FFFFH ; Result
        CLR     C
        MOV     A,#99H
        SUBB    A,B
        JC      END_PROCESS

        MOV     DPTR,#0000H ; Result
LOOP:   CLR     C
        MOV     A,DPL
        ADDC    A,B
        MOV     DPL,A
        MOV     A,DPH
        ADDC    A,#0
        MOV     DPH,A
        MOV     A,B
        XRL     A,#99H
        JZ      END_PROCESS
        INC     B
        JMP     LOOP

END_PROCESS:
        SJMP    $
        END
```

ค่า B ที่ทำให้คำตอบต่ำสุด คือ 99H(153)
ผลการบวก 99H(153) = 153(99H)
→ 1ไบต์

ค่า B ที่ทำให้คำตอบสูงสุด คือ 00
ผลการบวก 0+…+153 = 11781(2E05H)
→ 2ไบต์

Q8. ให้นักศึกษาเขียนโปรแกรมเพื่อหาค่าสูงสุดของข้อมูลดังต่อไปนี้

- ค่าตำแหน่งหน่วยความจำเริ่มต้นของบล็อกข้อมูลในการค้นหาเก็บไว้ที่ Address 20H
- ค่าตำแหน่งหน่วยความจำสุดท้ายของบล็อกข้อมูลในการค้นหาเก็บไว้ที่ Address 21H
- เก็บตำแหน่งหน่วยความจำที่พบค่าสูงสุดที่ R6
- เก็บค่าสูงสุดที่พบ ใน R7

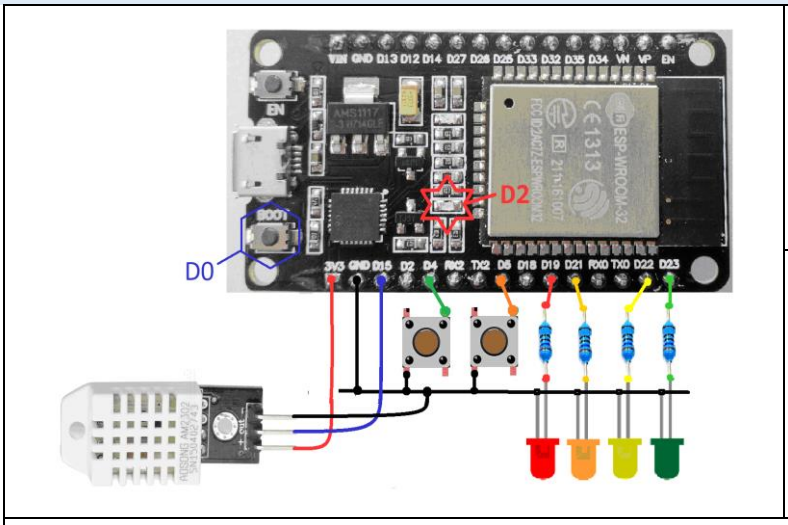
```
        ORG     0000H
        MOV     R7,#0H ; Start Max
        MOV     R0,20H ; Start Addr
        INC     21H    ; Stop+1

LOOP:   MOV     A,@R0
        CLR     C
        SUBB    A,R7
        JC      _XCHK
        MOV     A,R0   ; Address R6
        MOV     R6,A
        MOV     A,@R0  ; Value R7
        MOV     R7,A
_XCHK:  INC     R0
        MOV     A,R0
        CJNE    A,21H,LOOP

        SJMP    $
        END
```

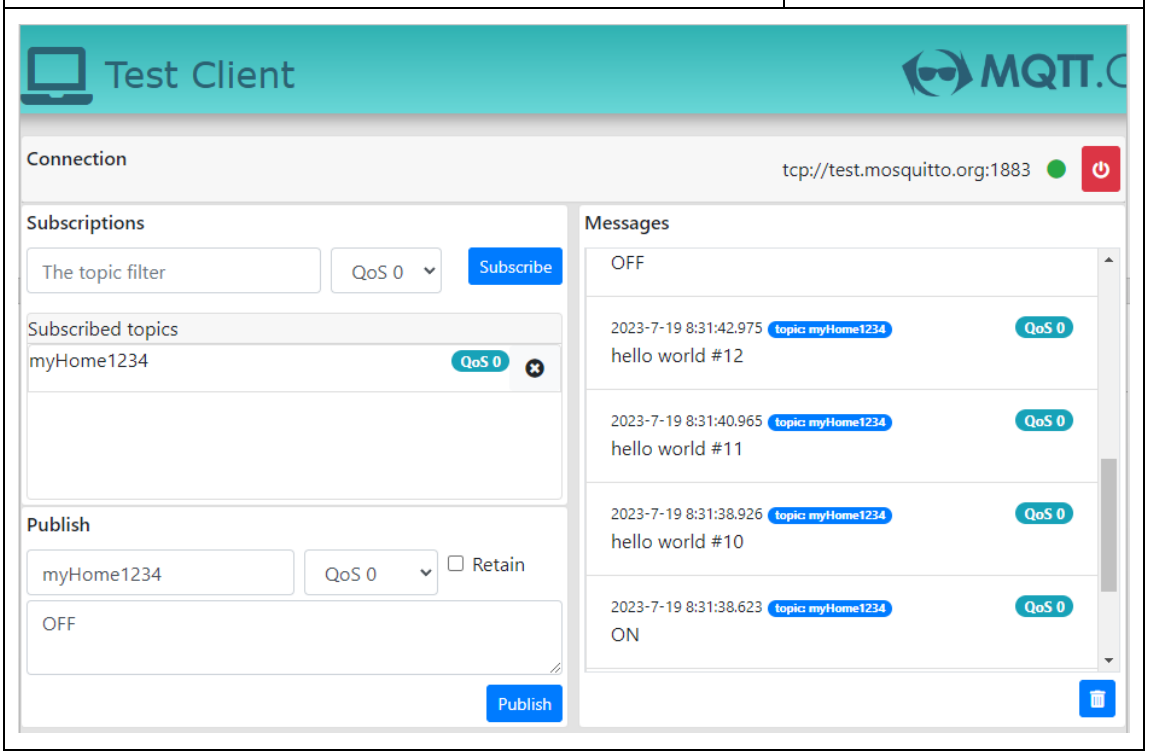
เริ่มจากค่าใน Addr.20H
ถึงค่าใน Addr.21H

หาก ตัวแปรX – ตัวแปรY แล้ว

- ไม่มีการยืม → X < Y
- มีการยืม → X >= Y

4/7. MQTT Protocol





https://testclient-cloud.mqtt.cool/

5/7. Ubidots IoT Broker

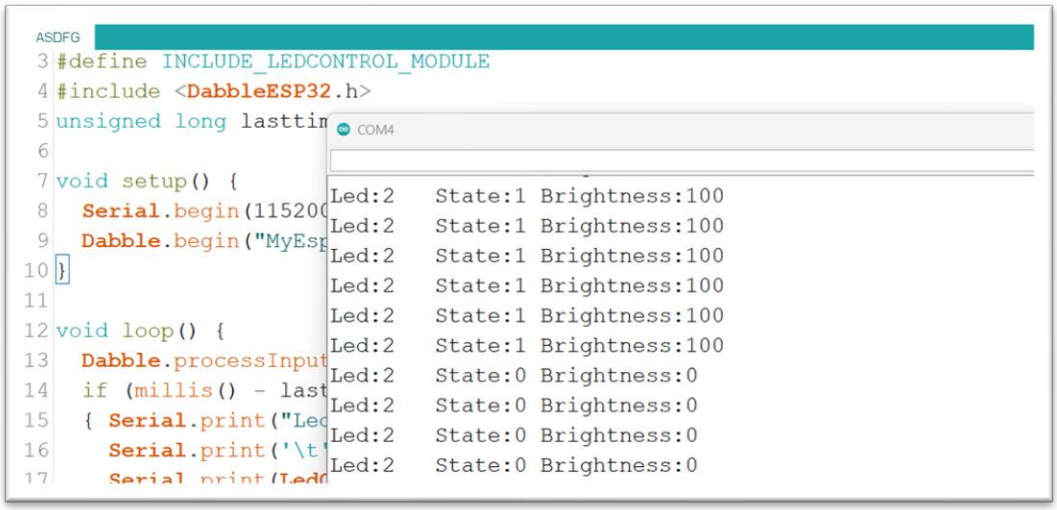
https://ubidots.com/

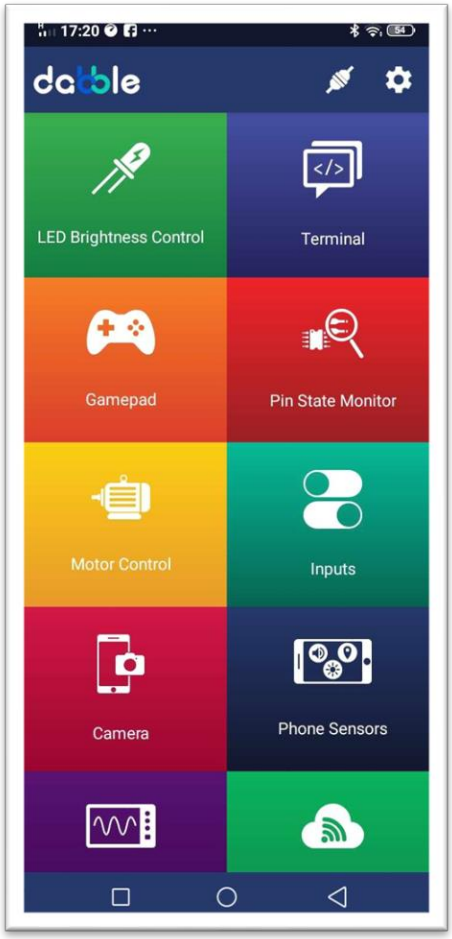
6/7. ESP32 – BLE Remote

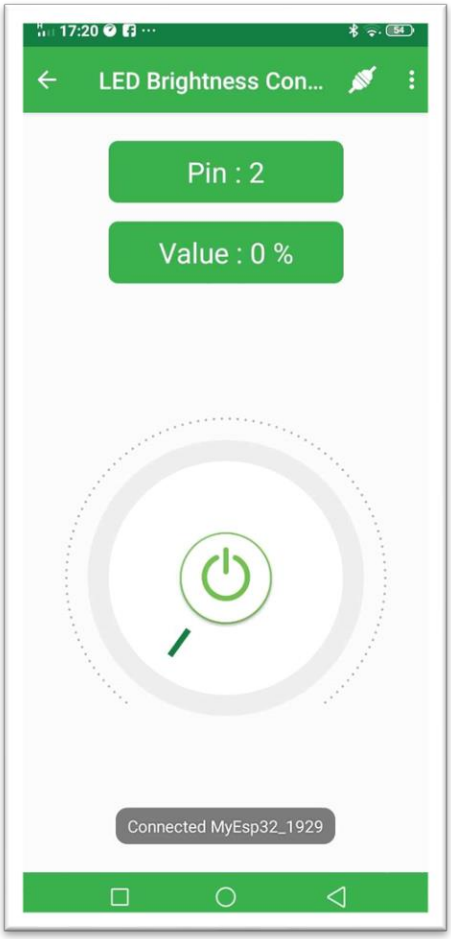
1. Read This: https://ai.thestempedia.com/docs/dabble-app/getting-started-with-dabble/
2. Install Mobile Application
   - Android: https://play.google.com/store/apps/details?id=io.dabbleapp
   - IOS: https://apps.apple.com/us/app/dabble-bluetooth-controller/id1472734455
3. Install ESP32 Library → DabbleESP32 by Mimansa Maheshwari Version 1.5.1

4. Test LED On/Off: File → Example → DabbleESP32 → 01LEDBrightControl

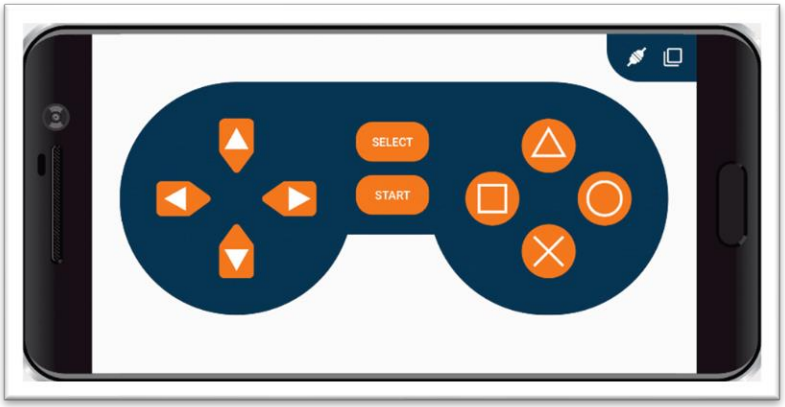
```
#define CUSTOM_SETTINGS
#define INCLUDE_LEDCONTROL_MODULE
#include <DabbleESP32.h>
unsigned long lasttime;

void setup() {
  Serial.begin(115200);
  Dabble.begin("MyEsp32_1929");
}

void loop() {
  Dabble.processInput();
  if (millis() - lasttime >= 250)
  { Serial.print("Led:"); Serial.print(LedControl.getpinNumber());
    Serial.print('\t');   Serial.print("State:");  // 0=Off. 1=On.
    Serial.print(LedControl.getpinState());
    Serial.print('\t');   Serial.print("Brightness:");
    Serial.println(LedControl.readBrightness());
    lasttime = millis();
  }
}
```




17:20
dabble
LED Brightness Control
Terminal
Gamepad
Pin State Monitor
Motor Control
Inputs
Camera
Phone Sensors

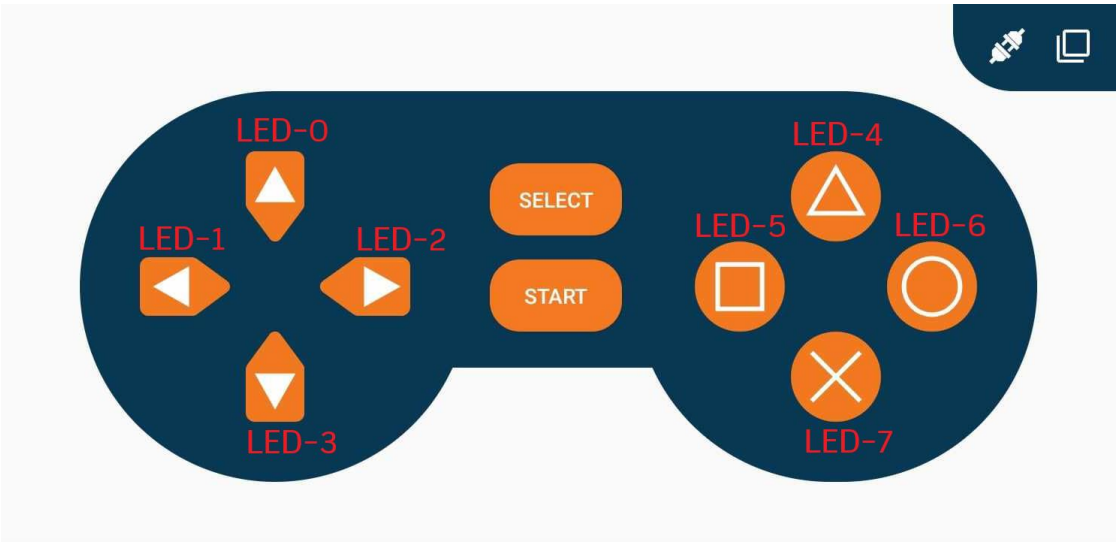

17:20
LED Brightness Con...
Pin : 2
Value : 0 %
Connected MyEsp32_1929

5. ทดสอบการใช้งาน Dabble Joystick




```
#define CUSTOM_SETTINGS
#define INCLUDE_GAMEPAD_MODULE
#include < DabbleESP32.h>
#define BLEDeviceName "MyEsp32_1929"
unsigned long lasttime;

void setup() {
  Serial.begin(115200);
  Dabble.begin(BLEDeviceName);
  Serial.println(BLEDeviceName);
}

void loop() {
  Dabble.processInput();
  if (millis() - lasttime >= 250) {
    lasttime = millis();
    Serial.print("\nKeyPressed: ");
    int Ang = GamePad.getAngle();
    int Rad = GamePad.getRadius();
    float XAis = GamePad.getXaxisData();
    float YAis = GamePad.getYaxisData();
    Serial.print("\tAngle: ");      Serial.print(Ang);
    Serial.print("\tRadius: ");     Serial.print(Rad);
    Serial.print("\tx_axis: ");     Serial.print(XAis);
    Serial.print("\ty_axis: ");     Serial.print(YAis);
    if (GamePad.isUpPressed())              Serial.print("\tUp");
    if (GamePad.isDownPressed())            Serial.print("\tDown");
    if (GamePad.isLeftPressed())            Serial.print("\tLeft");
    if (GamePad.isRightPressed())           Serial.print("\tRight");
    if (GamePad.isSquarePressed())          Serial.print("\tSquare");
    if (GamePad.isCirclePressed())          Serial.print("\tCircle");
    if (GamePad.isCrossPressed())           Serial.print("\tCross");
    if (GamePad.isTrianglePressed())        Serial.print("\tTriangle");
    if (GamePad.isStartPressed())           Serial.print("\tStart");
    if (GamePad.isSelectPressed())          Serial.print("\tSelect");
  }
}
```

6. ปรับปรุงโปรแกรมเพื่อควบคุม 8 LED แบบกดติดกดดับ ด้วย Dabble App



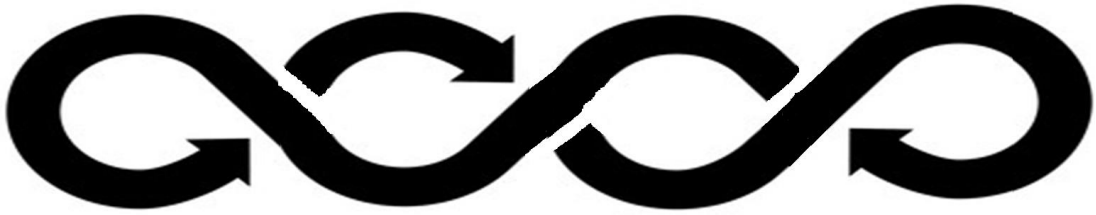

```
#define CUSTOM_SETTINGS
#define INCLUDE_GAMEPAD_MODULE
#include <DabbleESP32.h>
#define BLEDeviceName "MyEsp32_1929"
unsigned long lasttime;
#define pinTest1 15
#define pinTest2 2
int stsPinTest1 = 0;
int stsPinTest2 = 0;
bool chkPress = true;

void setup() {
  Serial.begin(115200);
  Dabble.begin(BLEDeviceName);
  Serial.println(BLEDeviceName);
  pinMode(pinTest1, OUTPUT);
  pinMode(pinTest2, OUTPUT);
}

void loop() {
  Dabble.processInput();
  digitalWrite(pinTest1, stsPinTest1);
  digitalWrite(pinTest2, stsPinTest2);

  if (millis() - lasttime > 250) {
    lasttime = millis();

    if (GamePad.isUpPressed()) {
      Serial.println("Key Up");
      stsPinTest1 = 1 - stsPinTest1;
      chkPress = true;
      while (chkPress) {
        delay(50);
        Dabble.processInput();
        chkPress = GamePad.isUpPressed();
      }
    }

    if (GamePad.isTrianglePressed()) {
      Serial.println("Key Triangle");
      stsPinTest2 = 1 - stsPinTest2;
      chkPress = true;
      while (chkPress) {
        delay(50);
        Dabble.processInput();
        chkPress = GamePad.isTrianglePressed();
      }
    }
  }
}
```

7/7. Robot with BLE Control

Read More

- https://www.robotique.tech/robotics/control-a-robot-car-based-on-esp32-by-bluetooth/

ภารกิจ

- ควบคุมรถด้วย BLE ผ่าน Mobile Application
- ให้วิ่งในสนามเป็นรูปลูป ดังรูป

- ตรงกลางของลูปจะใช้เก้าอี้วาง 3 ตัว (วางเหมือนกันกับงาน Week01, Week02)
- กำหนดให้มีช่องว่าง 1 ฟุต ระหว่างเก้าอี้แต่ละชุด
- กำหนดตำแหน่งคนคุมที่ด้านซ้ายของสนาม
- ปรับสนามเป็นรูป 4 ลูป